Tema 3: Condicionales

Sitio: CAMPUS
Curso: Python - 2º DAW
Libro: Tema 3: Condicionales
Imprimido por: MARÍA HAIZHEUSKAYA
Día: jueves, 29 de octubre de 2026, 21:44

Tabla de contenidos

1. Lógica Booleana
- 1.1. Expresiones Booleanas
- 1.2. Operadores Relacionales
- 1.3. Operadores Lógicos (and, or, not)
- 1.4. Comparaciones con Intervalos
- 1.5. Operador Módulo (%)

2. if y if-else
- 2.1. Sintaxis y Anatomía del "if"
- 2.2. Alternativa con if-else

3. if-elif-else y match-case
- 3.1. if-elif-else
- 3.2. match-case
- 3.3. if-elif-else vs. match-case

4. Condicionales Anidados
5. Operador Ternario
6. Buenas pácticas
7. Strings

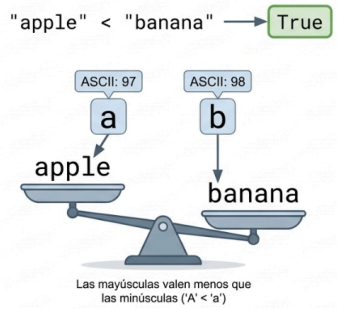
1. Lógica Booleana

1. Fundamentos de Lógica Booleana y Evaluación de Condiciones

La lógica booleana representa el cimiento matemático e informático esencial sobre el cual se erige la toma de decisiones en el software. Formalizada por el matemático británico George Boole a través del Álgebra de Boole, esta disciplina constituye la base de la aritmética computacional moderna. En la programación, un sistema informático no puede operar mediante ambigüedades; requiere evaluar enunciados para determinar su veracidad. Las estructuras condicionales se fundamentan en esta lógica para direccionar el flujo de ejecución del código, permitiendo que un programa no sea una secuencia lineal estática, sino un sistema dinámico capaz de reaccionar y adaptarse según los valores o el estado de los datos que procesa.

Operadores de Comparación				
1	==	(Igual que)	!=	(Diferente a)
2	>	(Mayor que)	<	(Menor que)
3	>=	(Mayor o igual)	<=	(Menor o igual)

El **Operador Módulo (%)**: Devuelve el residuo. Excelente para saber si un número es par: `x % 2 == 0`



1.1. Expresiones Booleanas

En Python, el almacenamiento y manejo de datos lógicos se realiza a través del tipo primitivo `bool`. Este tipo de dato solo puede tomar uno de dos valores posibles: `True` (Verdadero) o `False` (Falso). Es crucial destacar la sensibilidad a mayúsculas (*case sensitivity*): los identificadores `True` e `False` deben escribirse con la letra inicial en mayúscula; escribir `true` o `false` provocará un error de nombre (`NameError`) debido a que el intérprete no los reconoce como palabras reservadas o valores booleanos.

Una expresión booleana es cualquier evaluación sintáctica cuya resolución produce un valor del tipo `bool`. Por ejemplo, al evaluar operaciones aritméticas mediante operadores de comparación como `5 == 5`, la expresión devuelve `True`, mientras que `5 == 6` resulta en `False`.

Adicionalmente, Python posee la función incorporada de conversión de tipos `bool()`, la cual evalúa otros tipos de datos (como números o cadenas) y devuelve su valor de verdad correspondiente:

- **Tipos Numéricos (int, float)**: El número cero (`0` en enteros o `0.0` en flotantes) se evalúa como `False`. Cualquier valor numérico distinto de cero (positivo o negativo) se evalúa como `True`.
- **Cadenas de Texto (str)**: La cadena vacía (`""`) se evalúa como `False`. Cualquier cadena que contenga al menos un carácter (incluyendo un espacio en blanco `" "`) se evalúa como `True`.

Dato / Expresión Evaluada	Tipo Original	Resultado de bool()	Justificación Lógica
`0`	Entero (int)	**False**	El valor entero cero representa falsedad.
`0.0`	Flotante (float)	**False**	El cero en punto flotante representa falsedad.
`1` / `-5` / `3.14`	Entero / Flotante	**True**	Todo número distinto de cero representa veracidad.
`""`	Cadena (str)	**False**	La cadena vacía carece de contenido (falsa).
`"Texto"` / `" "`	Cadena (str)	**True**	Cadena no vacía (incluso un espacio posee caracteres).

1.2. Operadores Relacionales

Los operadores relacionales evalúan la relación entre dos operandos y devuelven un valor booleano (`True` o `False`). Python dispone de seis operadores relacionales principales:

- `==` : Igual a (verifica si dos valores son iguales).
- `!=` : No igual a / Diferente de (comprueba si dos valores son distintos).
- `>` : Mayor que.
- `<` : Menor que.
- `>=` : Mayor o igual que.
- `<=` : Menor o igual que.

Advertencia de Sintaxis: Un error común en etapas iniciales de aprendizaje es confundir el operador de asignación (`=`) con el operador relacional de igualdad (`==`). Mientras que `=` asigna un valor a una variable, `==` compara dos expresiones. Asimismo, es importante recordar que en Python no existen los símbolos `=<` ni `=>`; la sintaxis correcta exige colocar siempre el signo relacional antes del igual (`<=`, `>=`).

Cuando se aplican operadores relacionales a cadenas de texto, Python realiza una comparación lexicográfica. Este proceso compara las cadenas carácter por carácter, de izquierda a derecha, evaluando el valor numérico en su codificación estándar (ASCII / Unicode). Tan pronto como se encuentra una diferencia entre dos caracteres, se determina cuál cadena es "mayor" o "menor". Si las cadenas son idénticas pero una tiene más

caracteres (por ejemplo, `"apple"` y `"apples"`), la cadena más larga se considera mayor.

En la tabla ASCII/Unicode, las letras mayúsculas ('A' - 'Z') poseen valores numéricos inferiores a las letras minúsculas ('a' - 'z'). En consecuencia, 'a' (ASCII 97) es estrictamente mayor que 'A' (ASCII 65).

```
# Ejemplos de comparación relacional numérica y lexicográfica

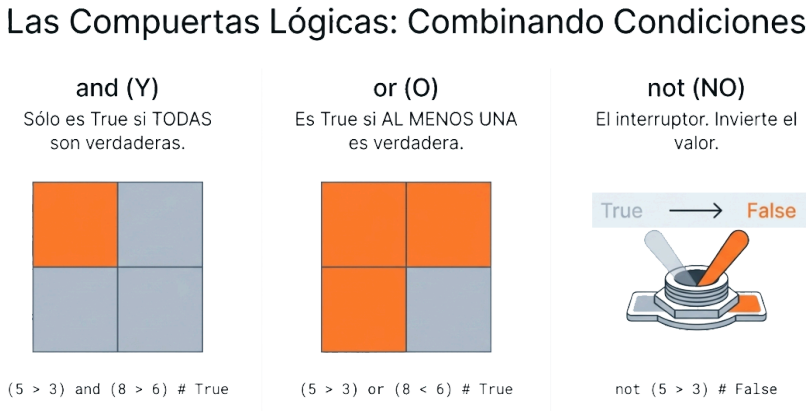
# Comparaciones numéricas simples
print(3 > 4)               # Devuelve: False
print(2 <= 4)              # Devuelve: True
print(2 != 22)             # Devuelve: True

# Comparación lexicográfica de cadenas
print("a" > "A")           # True: 'a' minúscula posee un valor Unicode/ASCII mayor que 'A'
print("apple" < "banana")  # True: 'a' tiene menor valor alfabético que 'b'
print("apple" > "Apple")   # True: 'a' minúscula es mayor que 'A' mayúscula
print("apple" < "apples")  # True: la primera cadena es más corta e idéntica en su inicio
print("Python" != "python")# True: difieren en la 'P'/'p' inicial
```

1.3. Operadores Lógicos (and, or, not)

Los operadores lógicos permiten evaluar expresiones compuestas agrupando múltiples condiciones booleanas.

- `and` **(Y lógico)**: Devuelve `True` únicamente si todas las condiciones evaluadas son verdaderas. Si una sola condición es `False`, toda la expresión resulta en `False`.
- `or` **(O lógico)**: Devuelve `True` si al menos una de las condiciones evaluadas es verdadera. Solo devuelve `False` cuando la totalidad de las condiciones son falsas.
- `not` **(NO lógico / Inversión)**: Es un operador unario que invierte el valor de verdad de una expresión booleana. Si se aplica a `True`, devuelve `False`, y viceversa.



A continuación, se presentan las tablas de verdad correspondientes a la combinación binaria de dos condiciones (p y q) mediante los operadores `and` y `or`, seguidas por la tabla del operador unario `not`:

Tabla 1: Operadores Lógicos Binarios (and, or)			
Condición 1 (p)	**Condición 2 (q)**	**p and q**	**p or q**
True	True	**True**	**True**
True	False	**False**	**True**
False	True	**False**	**True**
False	False	**False**	**False**

Tabla 2: Operador Lógico Unario (not)	
Condición (p)	**not (p)**
True	**False**

Tabla 2: Operador Lógico Unario (not)	
Condición (p)	**not (p)**
False	**True**

1.4. Comparaciones con Intervalos

Comparaciones Encadenadas (Estilo Matemático)

Una de las características más elegantes de Python, y que lo diferencia de lenguajes como Java o C++, es su capacidad para encadenar operadores de comparación de forma natural, exactamente igual a como se escriben en matemáticas.

En lugar de tener que usar operadores lógicos como `and` para unir dos condiciones separadas, Python permite evaluar un valor central contra dos límites simultáneamente. Esto hace que el código sea mucho más limpio y fácil de leer.

- **Sintaxis clásica (otros lenguajes):** `if temp >= 18.0 and temp <= 27.0:`
- **Sintaxis encadenada (Python):** `if 18.0 <= temp <= 27.0:`

Internamente, Python evalúa estas expresiones encadenadas expandiéndolas con un `and` implícito, pero con la ventaja de que **la variable central (en este caso `temp`) se evalúa una sola vez**, lo cual es más eficiente.

```
temp = 22.5

# Comprobando si un valor está dentro de un rango cerrado [18.0, 27.0]
if 18.0 <= temp <= 27.0:
    print("La temperatura es agradable.")
else:
    print("Hace demasiado frío o demasiado calor.")

# También se pueden encadenar operadores de igualdad
a = 10
b = 10
c = 10

if a == b == c:
    print("Todas las variables tienen el mismo valor.")
```

1.5. Operador Módulo (%)

El operador módulo, representado por el símbolo de porcentaje (`%`), trabaja sobre valores enteros (y expresiones enteras) devolviendo el residuo o resto de la división entera entre el primer operando y el segundo.

```
cociente = 7 / 3   # En Python 3 devuelve un número flotante: 2.3333333333333335
residuo = 7 % 3    # Devuelve 1, ya que 7 dividido entre 3 es 2 con residuo 1
```

Este operador posee aplicaciones fundamentales en la construcción de expresiones condicionales:

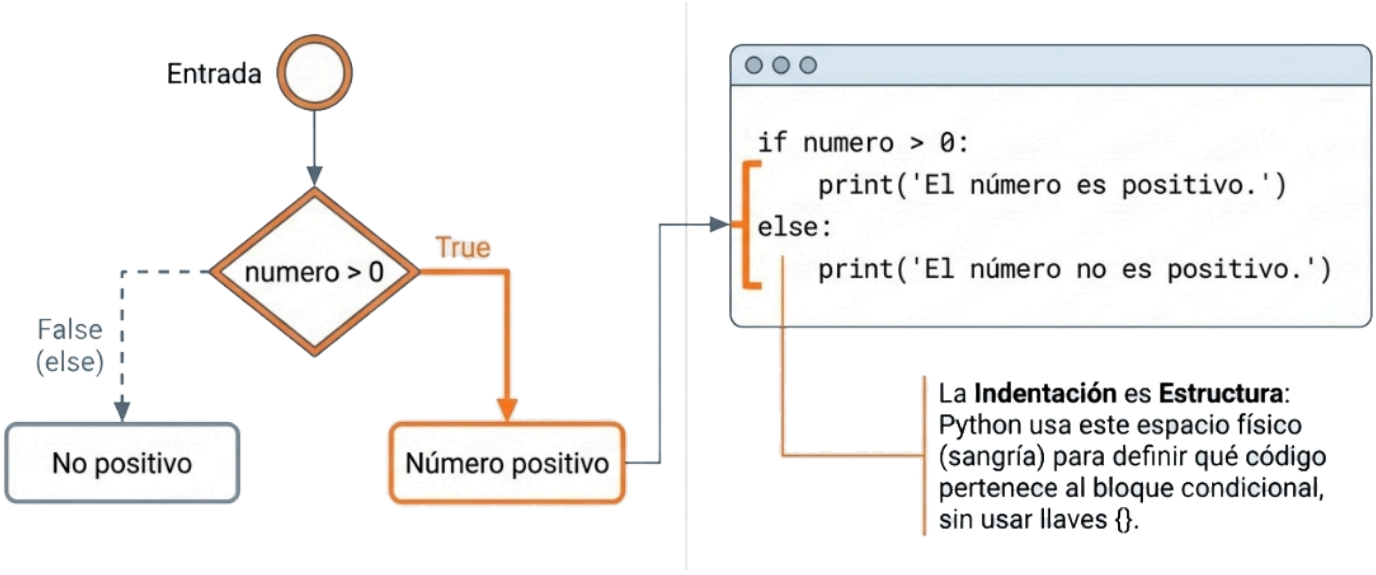
- **Evaluación de Paridad**: Un número entero `x` es par si el residuo de dividirlo entre 2 es igual a cero (`x % 2 == 0`). De lo contrario, es impar.
- **Comprobación de Divisibilidad**: Para determinar si un número `x` es divisible exactamente por otro número `n`, se evalúa si `x % n == 0`.
- **Extracción de Dígitos**: La operación `x % 10` extrae el dígito ubicado más a la derecha de un entero `x` (en base 10). Del mismo modo, `x % 100` extrae los dos últimos dígitos.

Comprendido cómo los operadores relacionales, lógicos y aritméticos generan y manipulan valores booleanos, el siguiente paso es integrarlos en estructuras que controlen la ejecución condicional primaria de un algoritmo.

2. if y if-else

Estructuras Condicionales Fundamentales: if y if-else

En su estado natural, un programa de computadora ejecuta sus instrucciones de manera secuencial, una tras otra, de arriba hacia abajo. Las estructuras condicionales rompen esta ejecución lineal plana, dotando al software de la capacidad de tomar caminos alternativos y adaptar su comportamiento en función del estado dinámico de las variables y datos procesados.



2.1. Sintaxis y Anatomía del "if"

La sentencia `if` es la forma más simple de control de flujo condicional. Se compone de un encabezado y un cuerpo:

- **Palabra reservada `if`**: Inicia la sentencia condicional.
- **Expresión booleana (Condición)**: Expresión evaluada inmediatamente después de `if`.
- **Dos puntos (`:`)**: Delimitan el final del encabezado e indican que el bloque subsiguiente está supeditado a la condición.
- **Cuerpo / Bloque indentado**: Una o más instrucciones sangradas que se ejecutarán únicamente si la condición evalúa a `True`.

```
if EXPRESION_BOOLEANA:
    SENTENCIAS
```

Reglas Estrictas de Indentación (Sangría)

En Python, el alcance del bloque de código no se delimita mediante llaves `{}` o palabras clave como `BEGIN/END`, sino a través de la indentación. La indentación es el espacio en blanco (recomendado formalmente a 4 espacios según PEP 8) al inicio de una línea de código.
Todas las sentencias con el mismo nivel de sangría después de un encabezado condicional forman parte del cuerpo del `if`. La primera línea no indentada marca el final explícito del bloque. Si la condición resulta verdadera (`True`), las sentencias del bloque se ejecutan secuencialmente; si resulta falsa (`False`), el bloque entero es ignorado.

La Instrucción pass

El cuerpo de un `if` debe contener obligatoriamente al menos una instrucción válida. Si por razones de diseño de software se requiere declarar una estructura condicional pero aún no se ha escrito el código del cuerpo, dejar el bloque vacío generará un error de sintaxis (`SyntaxError`). Para evitar esto, se utiliza la sentencia reservada `pass`, la cual actúa como un marcador de posición (*placeholder*) funcional que no realiza ninguna acción durante la ejecución.

```
if True:
    pass  # Marcador de posición funcional, evita el SyntaxError
```

Sintaxis en una Sola Línea

Para condicionales sumamente simples con una sola instrucción en el cuerpo, Python permite colocar la sentencia inmediatamente después de los dos puntos:

```
if a > 5: print("Es mayor que 5")
```

Si bien es sintácticamente válido, esta práctica solo se recomienda para casos extremadamente breves. Abusar de ella o colocar múltiples sentencias separadas por punto y coma (`;`) en la misma línea atenta contra la claridad y legibilidad del código.

2.2. Alternativa con if-else

Cuando se requiere manejar dos caminos excluyentes, la sentencia `if` se expande integrando la cláusula `else`. Esta estructura permite definir un "Plan B": la rama `if` se ejecuta si la condición es `True`, mientras que la rama `else` se ejecuta de forma automática e ineludible si la condición es `False`. Ambas ramas son mutuamente excluyentes; jamás se ejecutarán las dos dentro del mismo ciclo de evaluación.

```
# Ejemplo práctico: Verificación de paridad y mayoría de edad

def verificar_acceso(edad, numero_verificacion):
    # Evaluación de mayoría de edad (Plan A vs Plan B)
    if edad >= 18:
        print("Acceso concedido: Eres mayor de edad.")
    else:
        print("Acceso denegado: Eres menor de edad.")

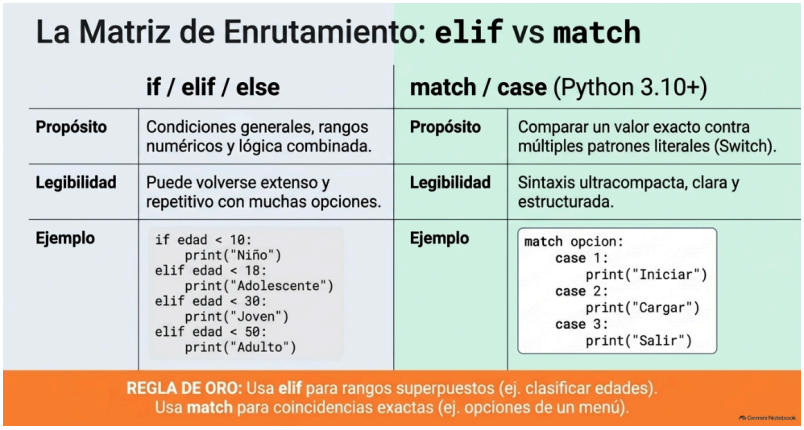
    # Evaluación de paridad utilizando el operador módulo
    if numero_verificacion % 2 == 0:
        print(f"El código {numero_verificacion} es PAR.")
    else:
        print(f"El código {numero_verificacion} es IMPAR.")

verificar_acceso(20, 15)
```

Existen escenarios complejos en la ingeniería de software donde dos alternativas (`if-else`) resultan insuficientes para representar la multiplicidad de estados posibles de un negocio o algoritmo.

3. if-elif-else y match-case

Gestionar múltiples escenarios de decisión de forma eficiente exige herramientas sintácticas que prevengan la redundancia de código y eviten una degradación en la legibilidad del programa cuando las posibilidades superan las dos ramas básicas.



3.1. if-elif-else

Para evaluar más de dos posibilidades de forma secuencial, Python provee la cláusula `elif`, una abreviatura de la expresión "else if".
En una estructura `if-elif-else`, las condiciones se evalúan de arriba hacia abajo en estricto orden sintáctico:

1. Se evalúa la condición del `if`. Si es `True`, se ejecuta su bloque y todo el resto de la estructura encadenada es ignorada.

2. Si la condición inicial es `False`, el flujo pasa al primer `elif`. Si este es `True`, se ejecuta su bloque y finaliza la estructura.
3. Si la condición del `elif` es `False`, se continúa evaluando los subsiguientes `elif` en secuencia.
4. La cláusula `else` es opcional y debe ubicarse siempre al final. Actúa como una rama por defecto cuando ninguna de las condiciones previas resultó verdadera.

```
# Ejemplo de clasificación escolar mediante condicionales encadenados
nota = 75

if nota >= 90:
    print("Excelente")
elif nota >= 75:
    print("Muy bien")
elif nota >= 60:
    print("Bien")
elif nota >= 40:
    print("Suficiente")
else:
    print("Insuficiente")
```

El uso de `elif` previene evaluaciones innecesarias. Una vez que se detecta la primera rama cuya condición sea verdadera, las ramas restantes son omitidas completamente por el intérprete.

3.2. match-case

Introducida formalmente en la versión 3.10 de Python, la estructura `match-case` proporciona una alternativa limpia, estructurada y expresiva al encadenamiento exhaustivo de bloques `elif` (similar al *switch-case* de otros lenguajes). Permite evaluar una variable o expresión contra múltiples patrones.

Anatomía Sintáctica de match-case:

- **`match variable:`** Declara el sujeto de evaluación.
- **`case valor:`** Define un patrón específico. Si el valor de la variable coincide exactamente con el patrón del case, se ejecuta su bloque asociado.
- **Comodín por defecto `case _:`** El guión bajo (_) actúa como un caso comodín que captura cualquier valor no cubierto anteriormente (equivalente al `else`).
- **Guardas condicionales (`case p if condición:`)**: Permiten añadir cláusulas de filtrado adicionales sobre el patrón evaluado.

Para comprender su progresión pedagógica, primero se examina la coincidencia directa de patrones literales (valores exactos) y, posteriormente, el uso de guardas condicionales complejas:

```
# 1. Ejemplo de coincidencia literal estricta (Patrón simple)
dia_numero = 3

match dia_numero:
    case 1:
        print("Lunes")
    case 2:
        print("Martes")
    case 3:
        print("Miércoles")
    case 4:
        print("Jueves")
    case 5:
        print("Viernes")
    case _:
        print("Día no válido o fin de semana")

# 2. Ejemplo avanzado: Clasificación con guardas condicionales (case p if condición)
peso = 12.5

match peso:
    case p if p <= 0:
        print("Peso no válido.")
    case p if p <= 5:
        print("Paquete pequeño.")
    case p if 5 < p <= 20:
        print("Paquete mediano.")
    case p if p > 20:
        print("Paquete grande.")
    case _:
        print("Entrada no reconocida.")
```

3.3. if-elif-else vs. match-case

Criterio de Evaluación	Estructura if-elif-else	Estructura match-case
Versión de Python requerida	Compatible con todas las versiones de Python.	Introducida exclusivamente a partir de Python 3.10+.
Propósito Principal	Resolver condiciones generales, rangos y combinaciones lógicas complejas.	Comparar una variable o patrón con múltiples opciones explícitas (pattern matching).
Legibilidad para Múltiples Casos	Puede volverse extensa y repetitiva si solo se comparan valores de una variable.	Alta claridad sintáctica; evita repetir el nombre de la variable en cada comparación.
Flexibilidad en Condiciones	Máxima flexibilidad para evaluar expresiones booleanas heterogéneas.	Excelente para concordancia de patrones de datos, secuencias y filtrado con guardas.

Aunque las estructuras avanzadas como `if-elif-else` organizan de forma elegante múltiples secuencias, la colocación sin control de condicionales dentro de otros condicionales genera problemas serios de arquitectura.

4. Condicionales Anidados

El anidamiento excesivo de código tiene un impacto sumamente negativo en la legibilidad y la mantenibilidad de un programa. Aumenta drásticamente la carga cognitiva necesaria para comprender el flujo de control, fenómeno comúnmente denominado "código en forma de flecha" (*arrow code*) debido a la acumulación progresiva de indentación hacia la derecha. Mantener la estructura del código lo más plana posible es una máxima fundamental en el desarrollo profesional.

4.1. Concepto y Estructura de Condicionales Anidados

Un condicional anidado es la práctica de colocar un bloque condicional (`if`, `if-else`) dentro del cuerpo de otra rama condicional previa.

```
# Ejemplo de condicionales anidados (Estructura jerárquica con sangría acumulada)
x = 7

if 0 < x:
    if x < 10:
        print("x es un número positivo de un solo dígito.")
```

En el fragmento anterior, para que la función `print()` sea ejecutada, el valor de `x` debe superar sucesivamente dos filtros independientes. La estructura exige mantener un seguimiento mental de dos niveles de profundidad en la indentación.

4.2. Estrategias de Refactorización y Simplificación

Los operadores lógicos (`and`, `or`) permiten aplanar y simplificar la mayoría de las estructuras anidadas en un único nivel de decisión.

Combina Expresiones Mediante and
Reemplazando el doble condicional por una expresión compuesta con el operador `and`:

```
# Refactorización con operador lógico
if 0 < x and x < 10:
    print("x es un número positivo de un solo dígito.")
```

Sintaxis Encadenada de Comparación (Estilo Matemático)
Python ofrece una característica sintáctica avanzada e intuitiva: la comparación encadenada directa. Permite traslapar operadores de comparación de forma análoga a la notación matemática tradicional (`0 < x < 10`).

```
# Refactorización con comparación encadenada al estilo matemático
if 0 < x < 10:
    print("x es un número positivo de un solo dígito.")
```

Las tres aproximaciones son exactamente equivalentes desde el punto de vista semántico; sin embargo, la comparación encadenada es estéticamente superior, más limpia y más rápida de interpretar por el desarrollador.
Junto a la simplificación de condicionales en bloque, existen casos donde se requiere una asignación o evaluación condicional compacta en una única línea de código.

5. Operador Ternario

Expresiones Condicionales Compactas (Operador Ternario)

El operador ternario ofrece una forma concisa de escribir una sentencia `if-else` en una sola línea. Esta técnica busca equilibrar la brevedad del código con la mantenibilidad, siendo ideal para asignaciones rápidas de datos basadas en una condición.

5.1. Sintaxis y Estructura del Operador Ternario

Formalizado en la propuesta PEP 308, el operador ternario en Python invierte el orden tradicional de los operandos en comparación con otros lenguajes de programación (como C o Java). Su sintaxis formal es:

```
[código_si_True] if [condición] else [código_si_False]
```

El flujo de evaluación ejecuta primero la condición central: si resulta `True`, retorna el valor o expresión ubicado a la izquierda (`código_si_True`); si resulta `False`, retorna la expresión ubicada a la derecha del `else` (`código_si_False`).

5.2. Casos de Uso Apropiados y Evaluaciones de Código

Asignación Dinámica de Variables
El operador ternario es altamente efectivo para asignar valores condicionales en una sola instrucción sin recurrir a un bloque `if-else` tradicional de cuatro líneas.

```
# Evita dividir por cero mediante una asignación previa condicional
a = 10
b = 5

c = a / b if b != 0 else -1
print(c)  # Devuelve 2.0 (si b fuera 0, asignaría -1)
```

Integración Directa en Llamadas a Funciones
Permite inyectar argumentos dinámicos dentro de llamadas a funciones como `print()`, haciendo el código más directo.

```
x = 6
# Evaluación de paridad e impresión inline
print("Es par" if x % 2 == 0 else "Es impar")
```

Límites de Uso y Mantenibilidad
A pesar de su practicidad, el operador ternario debe evitarse cuando la condición a evaluar o las expresiones de retorno son complejas, o cuando se pretende anidar ternarios dentro de otros ternarios. Forzar código complejo en una sola línea crea instrucciones crípticas, reduce la legibilidad y contraviene el diseño limpio del código.
Una vez dominada la condensación sintáctica de los condicionales, resulta indispensable analizar cómo se integran estas estructuras de control con las técnicas de saneamiento de datos de entrada y los estándares profesionales de codificación.

6. Buenas pácticas

Buenas Prácticas, Limpieza de Datos y Legibilidad del Código

Escribir código profesional implica no solo lograr que el programa ejecute sus tareas correctamente, sino garantizar que sea robusto frente a entradas no válidas, fácil de leer y sencillo de mantener por otros programadores. La interacción entre el saneamiento de datos y el flujo condicional determina la estabilidad de una aplicación en producción.

6.1. Integración de Métodos de Cadenas para Validación Previa

Las entradas provistas por usuarios a través de la terminal o formularios (mediante la función `input()`) se reciben invariablemente como cadenas de texto (`str`). Estas entradas suelen contener errores involuntarios como espacios accidentales o formatos incorrectos de caja (mayúsculas/minúsculas). Es vital sanitizar y validar el texto antes de ingresarlo a la lógica condicional de negocio.

Métodos de Manipulación Indispensables:

- `.strip()`: Elimina espacios en blanco sobrantes al inicio y al final de la cadena.
- `.lower()` / `.upper()`: Normalizan el texto a minúsculas o mayúsculas respectivamente, permitiendo realizar comparaciones insensibles a la caja (*case-insensitive*).
- `.isdigit()`: Devuelve `True` si todos los caracteres de la cadena son dígitos numéricos, lo que permite verificar la validez de un dato antes de forzar la conversión de tipo con `int()`.

```
"""
Caso práctico de saneamiento, validación previa de datos y verificación de tipos.
Diseñado para la validación de entrada de usuarios en aplicaciones interactivas.
"""

nombre_input = input("Ingrese su nombre: ")
edad_input = input("Ingrese su edad: ")
email_input = input("Ingrese su correo electrónico: ")

# Sanitización previa
nombre_limpio = nombre_input.strip()
edad_limpia = edad_input.strip()
email_limpio = email_input.strip().lower()

# Comprobación de campos vacíos, tipo numérico estricto y formato de correo
if len(nombre_limpio) == 0 or email_limpio == "":
    print("ERROR: Los campos no pueden estar vacíos.")
elif not edad_limpia.isdigit():
    print("ERROR: La edad ingresada debe ser un número entero válido.")
elif email_limpio.count("@") != 1 or " " in email_limpio:
    print("ERROR: El correo electrónico ingresado no es válido.")
else:
    edad_num = int(edad_limpia)
    print(f"Registro exitoso para: {nombre_limpio.title()} ({edad_num} años) - {email_limpio}")
```

6.2. Documentación y Comentarios Efectivos (# y """)

Los comentarios son herramientas primordiales para documentar decisiones de diseño lógico dentro de la base de código.

Tipos de Comentarios:

- **Comentarios de una línea (`#`)**: Anteponen el símbolo `#` a la instrucción. Son útiles para anotaciones breves y explicaciones de contexto sobre variables o líneas específicas.
- **Cadenas multilínea (`"""..."""` / `'''...'''`)**: Cadenas triples de texto que, cuando no están asignadas a una variable, actúan en la práctica como comentarios multilínea. Son recomendables para documentar módulos, describir funciones (*docstrings*) o detallar algoritmos extensos.

Reglas para el Uso Profesional de Comentarios:

- **Explicar el PORQUÉ, no el QUÉ**: Evite comentarios redundantes que solo reescriban la sintaxis obvia (ejemplo incorrecto: `x = 5 # Asignamos 5 a x`). Escriba comentarios que justifiquen decisiones (ejemplo correcto: `x = 5 # Cantidad mínima en stock antes de disparar reorden`).
- **Mantener la Sincronización**: Si modifica el código condicional, actualice de inmediato sus comentarios. Los comentarios desactualizados contradicen la lógica real del programa y generan confusión.

6.3. Principios de Legibilidad y Código Pythonico (PEP 8)

La guía oficial de estilo PEP 8 promueve la adopción de código legible, elegante y uniforme ("código pythonico"):

- **Consistencia de Indentación**: Utilice de forma estricta un mismo estándar de sangría en todo el script (preferentemente 4 espacios por nivel de indentación en lugar de tabuladores aislados).
- **Evitar Condicionales Booleanos Redundantes**: Al evaluar variables que ya contienen un valor booleano, no compare explícitamente contra `True` o `False`.
  Forma incorrecta / redundante: `if es_valido == True:`
  Forma correcta (pythonica): `if es_valido:`
  Para comprobar falsedad: `if not es_valido:`
- **Salidas Tempranas mediante la Sentencia return**: En lugar de envolver todo el cuerpo de una función en condicionales anidados profundos, evalúe las condiciones de error al inicio e interrumpa la ejecución inmediatamente con un `return` (patrón de cláusula de guarda).

```
# Ejemplo de salida temprana para manejo limpio de errores
def calcular_raiz_cuadrada(x):
    # Detección temprana de condición de error (Guard Clause)
    if x <= 0:
        print("Error: Por favor, use solo números positivos.")
        return  # Sale inmediatamente de la función

    # Flujo principal limpio de anidamiento
    resultado = x ** 0.5
    print(f"La raíz cuadrada es: {resultado}")
```

El dominio cabal de la lógica booleana, la estructuración pulcra de condicionales (`if`, `elif`, `match-case`), la simplificación de patrones anidados y la constante aplicación de normas de estilo legibles facultan al desarrollador para construir aplicaciones informáticas altamente interactivas, robustas, eficientes y libres de errores.

7. Strings

En Python, las cadenas de texto (*strings*) son objetos inmutables. Esto significa que una vez creadas, no se pueden modificar directamente. Sin embargo, Python proporciona una gran cantidad de **métodos** que realizan operaciones sobre la cadena original y nos devuelven una nueva cadena (o un valor booleano) como resultado.

1. Transformación de formato

Estos métodos son ideales para estandarizar textos, por ejemplo, antes de guardarlos en una base de datos o compararlos.

- `.lower()`: Convierte todos los caracteres de la cadena a minúsculas.
- `.upper()`: Convierte todos los caracteres a mayúsculas.
- `.capitalize()`: Convierte solo la primera letra de la cadena a mayúscula y el resto a minúsculas.

```
texto = "hOlA MuNdO"

print(texto.lower())       # "hola mundo"
print(texto.upper())       # "HOLA MUNDO"
print(texto.capitalize())  # "Hola mundo"
```

2. Limpieza de espacios

Muy útiles cuando recibimos datos introducidos por el usuario y queremos evitar errores por espacios accidentales al principio o al final.

- `.strip()`: Elimina los espacios en blanco (o caracteres especificados) tanto al principio como al final de la cadena.
- `.lstrip()` y `.rstrip()`: Hacen lo mismo pero solo a la izquierda (*left*) o a la derecha (*right*) respectivamente.

```
usuario = "   juan_perez   "

print(usuario.strip())   # "juan_perez"
print(usuario.lstrip())  # "juan_perez   "
```

3. Validación de contenido

Estos métodos devuelven `True` o `False` y son perfectos para usar dentro de condicionales (`if`).

- `.isdigit()`: Devuelve `True` solo si todos los caracteres de la cadena son números (0-9).
- `.isalpha()`: Devuelve `True` si todos los caracteres son letras (no admite espacios ni números).
- `.isalnum()`: Devuelve `True` si es alfanumérico (solo letras y números, sin espacios ni símbolos especiales).

```
edad = "25"
nombre = "Ana Maria" # Tiene un espacio

print(edad.isdigit())   # True
print(nombre.isalpha()) # False (por el espacio en blanco)
```

4. Búsqueda y Reemplazo

Permiten analizar el contenido de un texto o modificar partes específicas del mismo.

- `.replace(viejo, nuevo)`: Reemplaza todas las apariciones de una subcadena por otra nueva.
- `.count(subcadena)`: Cuenta cuántas veces aparece una subcadena o letra dentro del texto.
- `.startswith(subcadena)` / `.endswith(subcadena)`: Comprueba si el texto empieza o termina con la subcadena indicada.

```
archivo = "reporte_ventas.pdf"
frase = "El perro ladra y el perro corre"

print(archivo.endswith(".pdf"))         # True
print(frase.replace("perro", "gato"))   # "El gato ladra y el gato corre"
print(frase.count("perro"))             # 2
```

5. División y Unión de cadenas

Cruciales para transformar textos en listas y viceversa (por ejemplo, al leer archivos CSV o líneas de datos).

- `.split(separador)`: Divide la cadena utilizando el separador indicado (por defecto es el espacio) y devuelve una lista con las partes.
- `separador.join(lista)`: Es el método opuesto. Toma una lista de cadenas y las une en un solo texto, colocando el separador entre ellas.

```
datos = "manzana,pera,plátano"
lista_frutas = datos.split(",")
print(lista_frutas)  # ['manzana', 'pera', 'plátano']

# Volver a unir con un guion
texto_unido = " - ".join(lista_frutas)
print(texto_unido)   # "manzana - pera - plátano"
```